#3

Two Sum

Arrays & Hashing | LeetCode 1 | NeetCode "Two Sum"

PROBLEM

Given an array of integers `nums` and an integer `target`, return the indices `i` and `j` such that `nums[i] + nums[j] == target` and `i != j`. You may assume that every input has exactly one pair of indices that satisfies the condition. Return the answer with the smaller index first.

```
nums = [3,4,5,6], target = 7 → [0,1]
```

```
nums = [4,5,6], target = 10 → [0,2]
```

```
nums = [5,5], target = 10 → [0,1]
```

CORE IDEA

Goal

Find two indices whose values add up to the target.

Key insight

Rearranging `nums[i] + nums[j] == target` gives `nums[j] == target - nums[i]`. A hash map lets you check in O(1) whether the complement you need has already been seen, so one pass is enough — no need to check every pair.

> Trigger: “Which other number do I need to hit the target?” → store what you've seen in a Hash Map and look up the complement.

OPTIMAL SOLUTION

`Data structure` — Hash Map (Python `dict`) mapping value → index

1. Initialize an empty dict `result`.
2. Iterate through `nums` with index `i`.
3. Compute `diff = target - num` — the value that would complete the pair.
4. If `diff` is already a key in `result`, return `[result[diff], i]` immediately.
5. Otherwise, store `result[num] = i` and continue.

O(n)	O(n)
TIME	SPACE

CODE

```python
class Solution:
    def twoSum(self, nums: List[int], target:
int) -> List[int]:
        result = {}
        for i, num in enumerate(nums):
            diff = target - num
            if diff in result:
                return [result[diff], i]
            result[num] = i
```

TIPS TO REMEMBER

`Mental shortcut` — “What's missing? Look it up, don't loop for it.”

`Common / related` — Two Sum II - Input Array Is Sorted (two-pointer variant), 3Sum, 4Sum, Two Sum Less Than K

> *"I'm trading O(n) space for O(n) time by storing each number's index in a hash map as I go, so I can check for its complement in O(1) instead of nested-looping over every pair."*

EDGE CASES

- Negative numbers and zero — `diff = target - num` works the same regardless of sign.
- The same index can't be reused as both `i` and `j`, since the lookup happens before the current number is added to the map.
- Duplicate values, e.g. `nums = [5,5], target = 10` — the first 5 is stored before the second 5 looks up its complement, so `i != j` is naturally satisfied.
- Problem guarantees exactly one solution exists, so there's no need to handle a ‘no pair found’ case.